Design and Implementation of a Simple Query Processor

Krishna Singha	Shankit Kumar Das	Sukumar Singh	Suvadip Bhuiya	Aman Tudu
23CS10034	23CS10066	23CS10069	23CS10070	23CS30004

Abstract

This project presents the design and implementation of a compact SQL query processing engine that transforms user queries into executable operations over structured data. The system parses SQL statements into an internal representation, constructs a logical execution plan, and evaluates it over relations stored in CSV files with metadata-driven schemas. It supports essential database operations such as selection, projection, joins, grouping, and data modification.

The architecture is modular, separating parsing, planning, execution, and storage, enabling clarity of design and ease of extension. While optimization is currently lightweight, the system incorporates practical strategies such as early filtering and efficient execution pipelines. Overall, the implementation demonstrates a complete end-to-end query processing workflow and provides a strong foundation for future enhancements in cost-based and rule-based optimization.

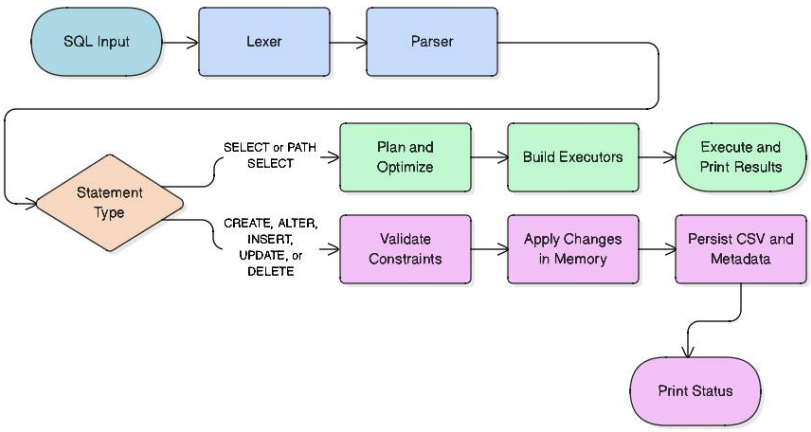

Figure 1: Query Processing Pipeline

1. System Features and Capabilities

The implemented system supports a wide range of SQL query processing functionalities, summarized below.

1.1. Core SQL Operations

`SELECT`, `CREATE TABLE`, `ALTER TABLE`, `INSERT`, `UPDATE`, `DELETE`.

1.2. Query Clauses

- **Filtering:** `WHERE`, `HAVING`
- **Projection control:** `DISTINCT`
- **Joins:** `INNER`, `LEFT`, `RIGHT`, `FULL`, `CROSS`
- **Aggregation:** `GROUP BY`
- **Ordering:** `ORDER BY` (`ASC`, `DESC`)
- **Result limiting:** `LIMIT`

1.3. Aggregation Functions

`COUNT(*)`, `COUNT(a)`, `COUNT DISTINCT`, `SUM`, `AVG`, `MIN`, `MAX`.

1.4. Schema Management

Column operations include add, drop, rename, and datatype modification. Constraint handling is integrated within schema updates.

1.5. Constraints and Data Integrity

`PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `FOREIGN KEY`, `CHECK`.

1.6. Supported Data Types

- **Numeric:** `INT`, `FLOAT`, `DOUBLE`
- **Text:** `VARCHAR`, `TEXT`, `ENUM`

- **Others:** `BOOLEAN`, `TIMESTAMP`

1.7. Execution and Optimization Features

- Predicate pushdown and filter simplification.
- Projection pruning to reduce intermediate data.
- Adaptive join strategy selection.
- Index-aware equality access (in-memory hash indexing).
- Scoped cost-based join reordering.

2. Methodology

The methodology of this project focuses on building a clear and traceable SQL query processing pipeline that transforms raw query text into executable operations over structured data. The system is designed with modular layers, where each stage — input handling, parsing, planning, optimization, and execution — has a well-defined responsibility. This separation improves clarity, maintainability, and ease of testing.

The current implementation prioritizes correctness and deterministic behavior, while incorporating lightweight optimization techniques where beneficial. This approach ensures a reliable and understandable system, while providing a solid foundation for future extensions such as cost-based optimization and advanced query rewriting.

2.1. Data Storage and Metadata Management

The system uses a simplified storage model where table data is stored in CSV files, while schema information is maintained separately in a metadata catalog (JSON format). Each CSV file represents a relation with rows as tuples and columns as attributes, enabling easy access and integration with the execution engine.

The metadata layer defines table structure, including attribute names, data types, and constraints such as `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `FOREIGN KEY`, and `CHECK`. This separation between data and schema ensures that raw values can be interpreted correctly during query processing.

At startup, both data and metadata are loaded into memory, allowing efficient query execution and fast validation without repeated disk access. This design keeps the system lightweight while

maintaining consistency and correctness across operations.

2.2. Query Input Processing and Statement Handling

This stage is responsible for acquiring SQL input and preparing it for structured processing. Queries are typically read from a script file (e.g., `queries.sql`) or provided directly to the system. Since input may contain multiple statements, comments, and varying formats, preprocessing is required to ensure correct interpretation.

The system splits the input into individual statements using semicolon delimiters while carefully handling semicolons within string literals. Comments are removed and string contents are preserved to maintain query semantics. Additionally, line and column positions are tracked to enable precise error reporting.

Each statement is processed independently, ensuring that errors in one query do not affect others.

2.3. Lexical Analysis and Parsing

After preprocessing, each SQL statement is converted into a structured representation through lexical analysis and parsing.

- **Lexical Analysis (Lexer):** The input query is broken into tokens such as keywords, identifiers, literals, operators, and punctuation. Keywords are handled in a case-insensitive manner, and operators are normalized to ensure consistency.
- **Parsing:** The token stream is processed to construct an Abstract Syntax Tree (AST). The AST represents the logical structure of the query, including projections, table references, predicates, joins, grouping, and ordering clauses.

This structured representation enables the system to validate syntax, enforce operator precedence, and prepare queries for logical plan generation.

2.4. Execution Path Selection and Processing

Once a query is parsed into an AST, the system determines the appropriate execution path based on the type of statement.

SELECT Queries

- Construct a logical execution plan using relational operators.

- Apply lightweight optimization techniques such as predicate pushdown, filter simplification, projection pruning, and join strategy selection.
- Translate the logical plan into executor nodes and execute them to produce result tuples.

Data Definition and Manipulation Queries (CREATE, ALTER, INSERT, UPDATE, DELETE)

- Validate data types and enforce schema constraints.
- Apply modifications to in-memory data structures and schema definitions.
- Persist validated changes to CSV storage and the metadata catalog.

2.5. Enforcement of Correctness and Constraints

Before final execution or persistence, the system enforces correctness through type validation and constraint checking. This ensures that all operations maintain data integrity and adhere to schema definitions.

- **Type Validation:** Values are checked against supported data types such as `INT`, `VARCHAR`, `TEXT`, `BOOLEAN`, `FLOAT`, `DOUBLE`, `TIMESTAMP`, and `ENUM`.
- **Constraint Enforcement:** The system verifies constraints including `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `FOREIGN KEY`, and `CHECK` conditions.

All validation steps are performed prior to applying changes, ensuring that only consistent and valid data is written to storage.

2.6. Output Generation and Result Handling

After execution, the system generates output based on the type of query and presents it in a structured, user-readable format.

- **SELECT Queries:** Resulting tuples are formatted and displayed according to the projection specified in the query.
- **Data Modification Queries:** Operations such as `INSERT`, `UPDATE`, and `DELETE` return confirmation messages along with the number of affected rows.
- **Script Execution:** When multiple queries are executed in sequence, the system provides a summary of execution results including success status and error reporting.

3. Query Optimization Techniques

The system incorporates a hybrid optimization approach combining rule-based transformations with lightweight cost-based decision making. The goal is to improve execution efficiency while maintaining a manageable and transparent planning process.

3.1. Rule-Based Logical Optimizations

The optimizer applies a fixed sequence of rewrite rules over the logical plan:

- **Constant Folding and Filter Elimination:** Simplifies boolean expressions and removes redundant filters (e.g., `WHERE true`, `WHERE false`).
- **Filter Merging:** Combines stacked filter operations into a single conjunctive predicate to reduce evaluation overhead.
- **Predicate Pushdown:** Moves filter conditions closer to data sources. For joins, predicates are split into left and right components when safe, enabling early row elimination.
- **Projection Pruning:** Removes unused columns at the scan level by analyzing required attributes across the query plan, reducing intermediate data size.
- **Access Path Selection:** Replaces sequential scans with index-based access when predicates involve simple equality conditions on indexed columns.

3.2. Cost-Based and Adaptive Optimization

In addition to rule-based rewrites, the system integrates lightweight cost-based strategies:

- **Join Reordering:** Uses dynamic programming over join subsets to determine an efficient execution order for inner joins, guided by estimated cardinalities and predicate selectivities.
- **Cost Model:** Estimates selectivity and intermediate result sizes using statistics such as row counts and number of distinct values (NDV). This guides operator selection and join planning.
- **Join Algorithm Selection:** Chooses between nested loop, hash join, and merge join based on input size, join type, and data characteristics.

3.3. Execution-Level Optimizations

Additional optimizations are applied during execution:

- **Adaptive Join Execution:** The executor dynamically selects or adjusts join strategies based on runtime conditions such as input size and ordering.
- **Hash Index Caching:** In-memory hash indexes are built per table and reused across queries, with invalidation on data updates.
- **Join Micro-Optimizations:** Includes building hash tables on smaller inputs, skipping redundant predicate checks for equi-joins, and efficient merge join processing.

3.4. Statistics Support

The optimizer is supported by basic statistics collection, including table cardinality, distinct value counts, and numeric distributions. These statistics are used to estimate selectivity and guide cost-aware decisions.

4. Results and Screenshots

The system was evaluated using multiple SQL queries including selection, filtering, joins, aggregation, and data modification operations. The results demonstrate correct execution and consistent output formatting.

Figure 2: Simple Query Processor Terminal Interface

4.1. Basic Query Execution

```
BongoDB> SELECT user_id, username, dept_id
...> FROM users
...> ORDER BY user_id
...> LIMIT 5;
user_id | username | dept_id
--------+----------+--------
      1 | alice    |       1
      2 | bob      |       1
      3 | charlie  |       2
      4 | diana    |       2
      5 | ethan    |       3
(5 tuples)
```

```
BongoDB> SELECT username, dept_id
...> FROM users
...> WHERE dept_id IN (1,2,3)
...> ORDER BY username;
username | dept_id
---------+--------
alice    |       1
bob      |       1
charlie  |       2
diana    |       2
ethan    |       3
fatima   |       3
(6 tuples)
```

Figure 3: Basic `SELECT` query (left) and `WHERE` filtering (right)

4.2. Core SQL Operations

```
BongoDB> SELECT u.username, d.dept_name
...> FROM users u
...> INNER JOIN departments d
...> ON u.dept_id = d.dept_id
...> ORDER BY u.username
...> LIMIT 10;
username | dept_name
---------+------------
alice    | Engineering
bob      | Engineering
charlie  | Sales
diana    | Sales
ethan    | HR
fatima   | HR
george   | Finance
hana     | Finance
ivan     | Product
julia    | Product
(10 tuples)
```

```
BongoDB> SELECT dept_id, COUNT(*), AVG(user_id)
...> FROM users
...> GROUP BY dept_id
...> HAVING COUNT(*) >= 2
...> ORDER BY COUNT(*) DESC;
dept_id | COUNT(*) | AVG(user_id)
--------+----------+------------
      1 |        2 |         1.5
      2 |        2 |         3.5
      3 |        2 |         5.5
      4 |        2 |         7.5
      5 |        2 |         9.5
      6 |        2 |        11.5
      7 |        2 |        13.5
(7 tuples)
BongoDB> SELECT COUNT(*), SUM(score), AVG(score), MIN(score), MAX(score)
...> FROM events;
COUNT(*) | SUM(score) | AVG(score) | MIN(score) | MAX(score)
---------+------------+------------+------------+-----------
      15 |     1154.3 |  76.953333 |       58.8 |       95.0
(1 tuple)
```

Figure 4: `JOIN` operations (left) and `GROUP BY` aggregation (right)

```
BongoDB> SELECT u.username
...> FROM users u
...> WHERE EXISTS (
...>     SELECT e.enroll_id
...>     FROM enrollments e
...>     WHERE e.user_id = u.user_id
...> )
...> LIMIT 10;
username
--------
alice
bob
charlie
diana
fatima
george
hana
ivan
julia
kevin
(10 tuples)
```

```
BongoDB> SELECT DISTINCT dept_id
...> FROM users
...> ORDER BY dept_id;
dept_id
-------
      1
      2
      3
      4
      5
      6
      7
      8
(8 tuples)
```

Figure 5: `EXISTS` operations (left) and `DISTINCT` operations (right)

4.3. Data Modification and Advanced Queries

```
BongoDB> INSERT INTO transactions (id, title, amount)
...> VALUES (100, 'test', 500);
(1 row inserted)
BongoDB>
BongoDB> UPDATE transactions
...> SET amount = 600
...> WHERE id = 100;
(1 row updated)
BongoDB>
BongoDB> SELECT id, title, amount
...> FROM transactions
...> WHERE id = 100;
id  | title | amount
----+-------+-------
100 | test  |    600
(1 tuple)
```

```
BongoDB> CREATE TABLE demo_users (
...>     id INT PRIMARY KEY,
...>     name VARCHAR(50),
...>     age INT
...> );
Table 'demo_users' created.
BongoDB>
BongoDB> INSERT INTO demo_users VALUES (1, 'Rahul', 22);
(1 row inserted)
BongoDB> INSERT INTO demo_users VALUES (2, 'Aman', 24);
(1 row inserted)
BongoDB>
BongoDB> SELECT * FROM demo_users;
id | name  | age
---+-------+----
 1 | Rahul |  22
 2 | Aman  |  24
(2 tuples)
BongoDB>
```

Figure 6: `INSERT` and `UPDATE` operations (left); conditional query execution (right)

4.4. Discussion

The results confirm that the query processor correctly executes a wide range of SQL operations including filtering, joins, aggregation, and updates. The outputs are consistent with expected relational semantics, and constraint enforcement is handled effectively. The system demonstrates reliability and correctness across different query types while maintaining a clean and user-friendly interface.

References

1. Silberschatz, Korth, and Sudarshan. *Database System Concepts*. 7th Edition.
2. Raghu Ramakrishnan and Johannes Gehrke. *Database Management Systems*. 3rd Edition.
3. Jan Van den Bussche, Stijn Vansummeren. *Translating SQL into Relational Algebra*.
4. PostgreSQL Documentation: https://www.postgresql.org/docs/
5. SQLite Documentation: https://www.sqlite.org/docs.html
6. RFC 4180: Common Format and MIME Type for CSV Files: https://www.rfc-editor.org/rfc/rfc4180
7. TPC-H Benchmark Documentation: https://www.tpc.org/tpch/

Github Link

1. https://github.com/suvadipbhuiya2004/simple-query-processor.git